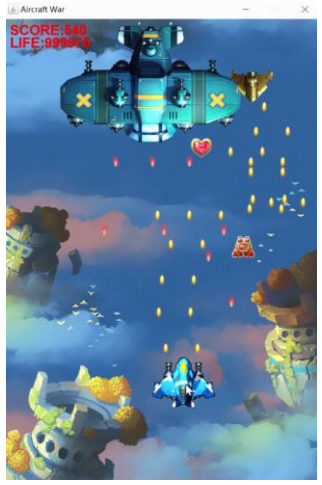
实验六：观察者和模板模式

实验与创新实践教育中心 • 计算机与数据技术实验教学部

本学期实验总体安排

初始版本

最终版本



初始版本

游戏主界面
英雄机移动
英雄机子弹直射
碰撞检测
统计得分和生命值

01

UML建模
精英敌机生成、攻击
道具生成、加血道具生效
采用**单例模式**创建英雄机

02

采用**简单工厂模式**
创建道具
采用**工厂模式**
创建敌机

03

采用**策略模式**
实现不同飞机的发射弹道
两种火力道具生效

04

采用**数据访问对象模式**
实现得分排行榜
添加**JUnit单元测试**

05

使用**Swing**添加游戏难度选择和排行榜界面
使用**多线程**实现音效控制及火力道具状态恢复

06

采用**观察者模式**
实现炸弹、冰冻道具
采用**模板模式**
实现三种游戏难度

本学期实验总体安排

实验项目	一	二	三	四	五	六
学时数	2	2	2	2	4	4
实验内容	系统分析 单例模式	简单工厂 工厂模式	策略模式	JUnit 数据访问 对象模式	Swing 多线程	观察者模式 模板模式
时间节点			**中期**			**结项**
提交内容			UML类图 项目代码			实验报告 代码、类图 展示视频（选）

实验课程共 16 个学时，6 个实验项目，总成绩为 30分 。

目录

CONTENTS

01 实验目的

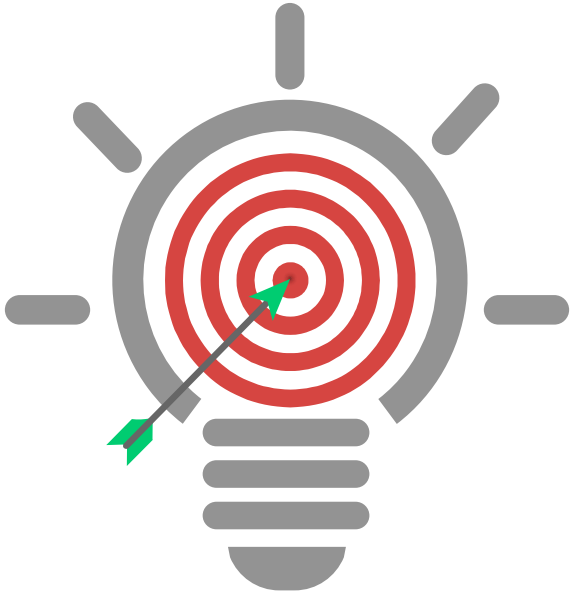
02 实验任务

03 实验步骤

04 期末提交

实验目的

- 深入理解**观察者**和**模板模式**的设计意图及结构
- 结合案例，规范绘制两种模式的**UML类图**
- 掌握**重构**方法，通过**代码实现**并应用两种设计模式



实验任务

1. 深入学习**观察者**与**模板模式**的设计原理，结合实例完成两种模式的**UML结构图**绘制
2. 对系统内的道具生效逻辑与难度控制逻辑进行代码重构：
   - 采用观察者模式实现炸弹、冰冻道具的功能逻辑
   - 采用模板模式实现简单、普通和困难三种游戏难度
3. 完成本次迭代开发清单中的全部任务

实验步骤：炸弹、冰冻道具场景分析（1）

炸弹、冰冻道具场景分析

游戏中包含2种特殊道具：**炸弹**道具、**冰冻**道具。

英雄机碰撞后自动触发生效，炸弹、冰冻道具对**敌机子弹**及**不同类型敌机**的效果存在**显著差异**：

	炸弹道具	冰冻道具
普通敌机	坠毁	永久静止
精英敌机	坠毁	静止4s后恢复
精锐敌机	坠毁	静止3s后恢复
王牌敌机	掉血	减速5s后恢复
Boss敌机	不受影响	不受影响
敌机子弹	消失	静止5s后恢复

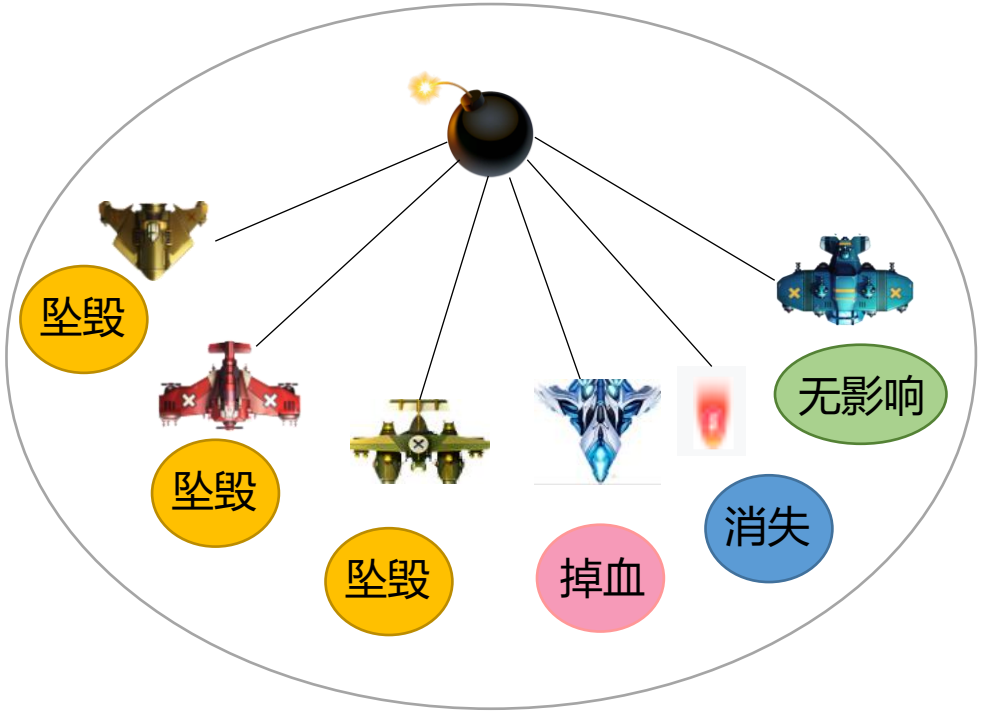
注：炸弹道具生效后，英雄机可获得所有坠毁敌机分数， 坠毁敌机不掉落道具。

实验步骤：炸弹、冰冻道具场景分析（2）

请思考：

1. 目前在哪个类实现炸弹爆炸？有哪些角色对炸弹有响应？如何响应？

```java
public class BombSupply extends AbstractFlyingSupply{
    public BombSupply(int locationX, int locationY, int speedX, int speedY) {
        super(locationX, locationY, speedX, speedY);
    }

    @Override
    public void activate() {
        System.out.println("BombSupply active!");
    }
}
```

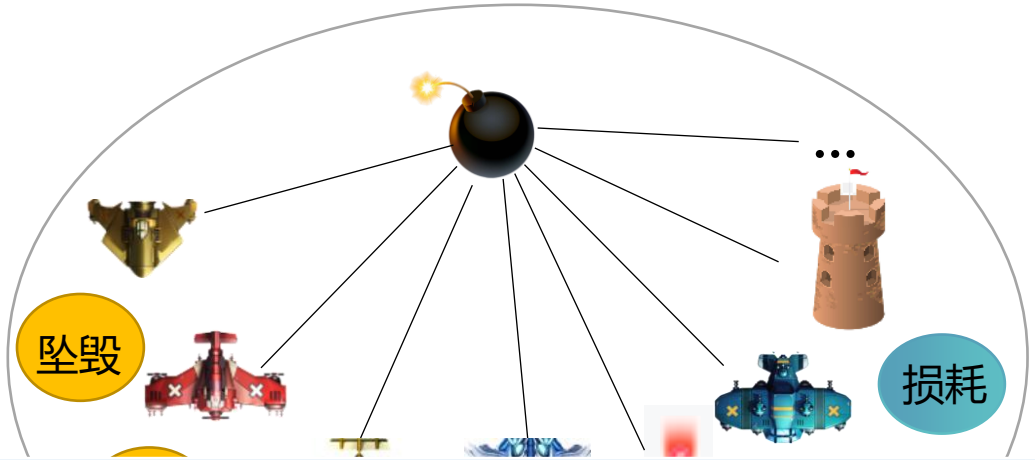


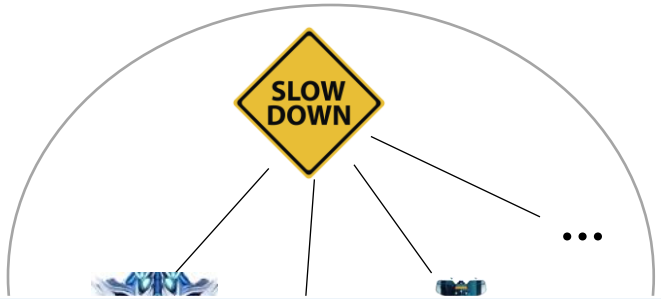
实验步骤：炸弹、冰冻道具场景分析（3）

请思考：

2. 如何增加一个对炸弹有响应的新角色，如敌方炮台？

3. 如何增加冰冻道具？





违反
单一职责 X

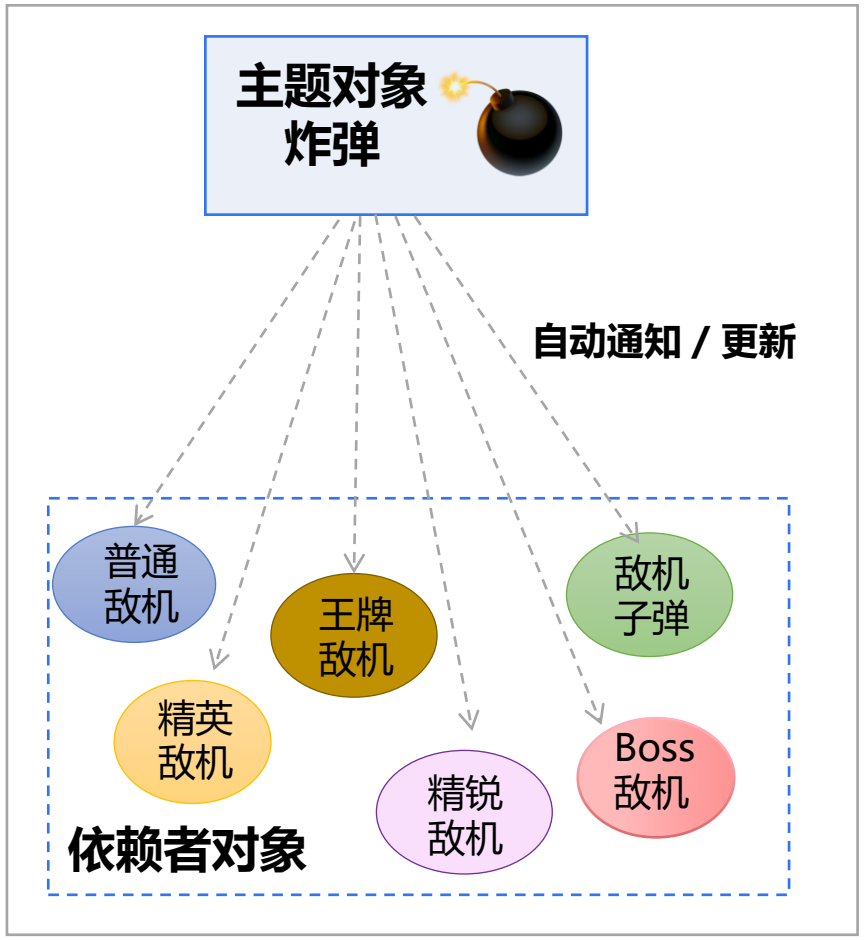
违反
开闭原则 X

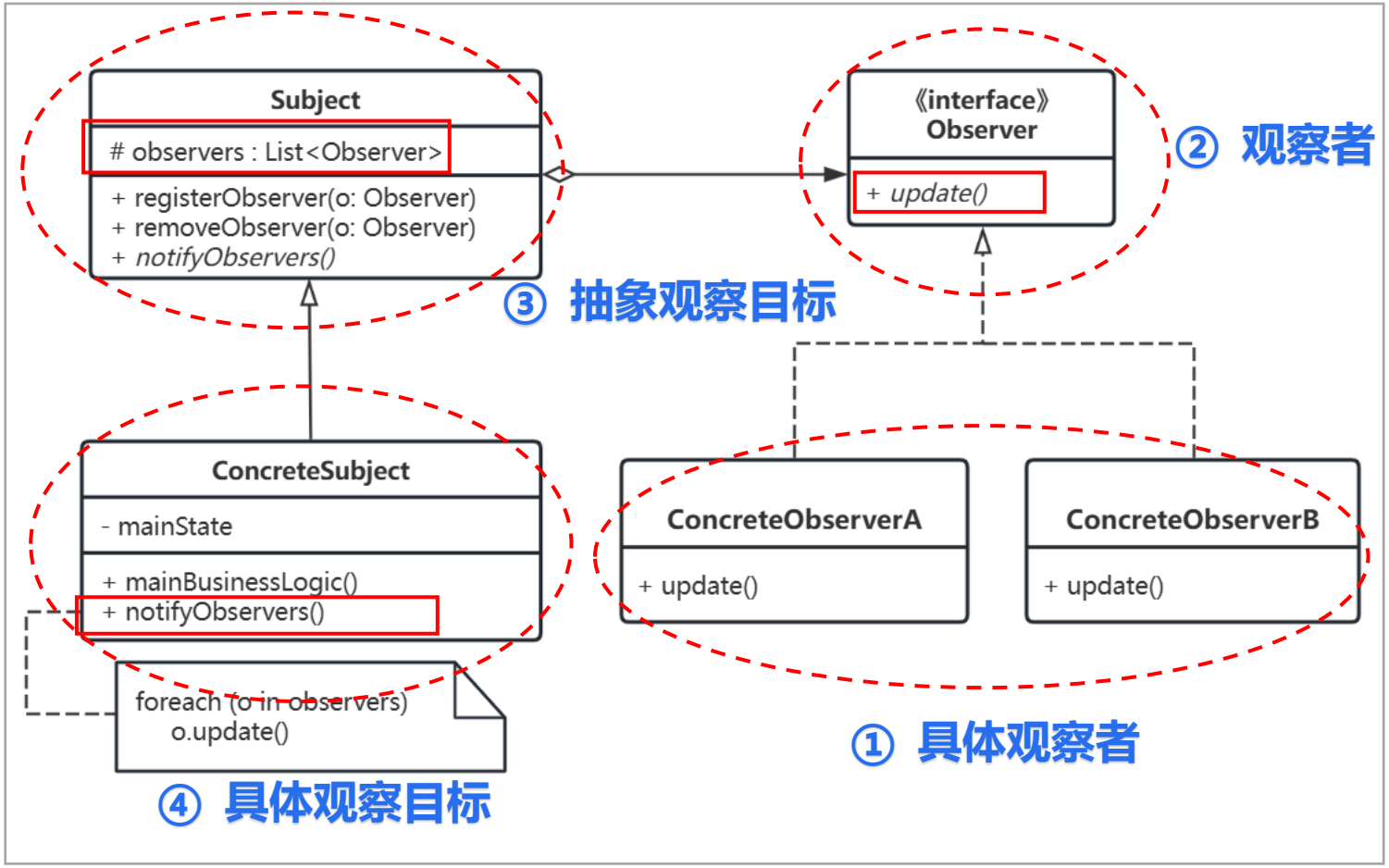
违反
X

针对接口编程，而不是针对实现编程！

实验步骤：观察者模式结构图

观察者模式（Observer Pattern）定义了对象之间的一对多依赖。当一个对象改变状态时，它的所有依赖者都会收到通知并自动更新。



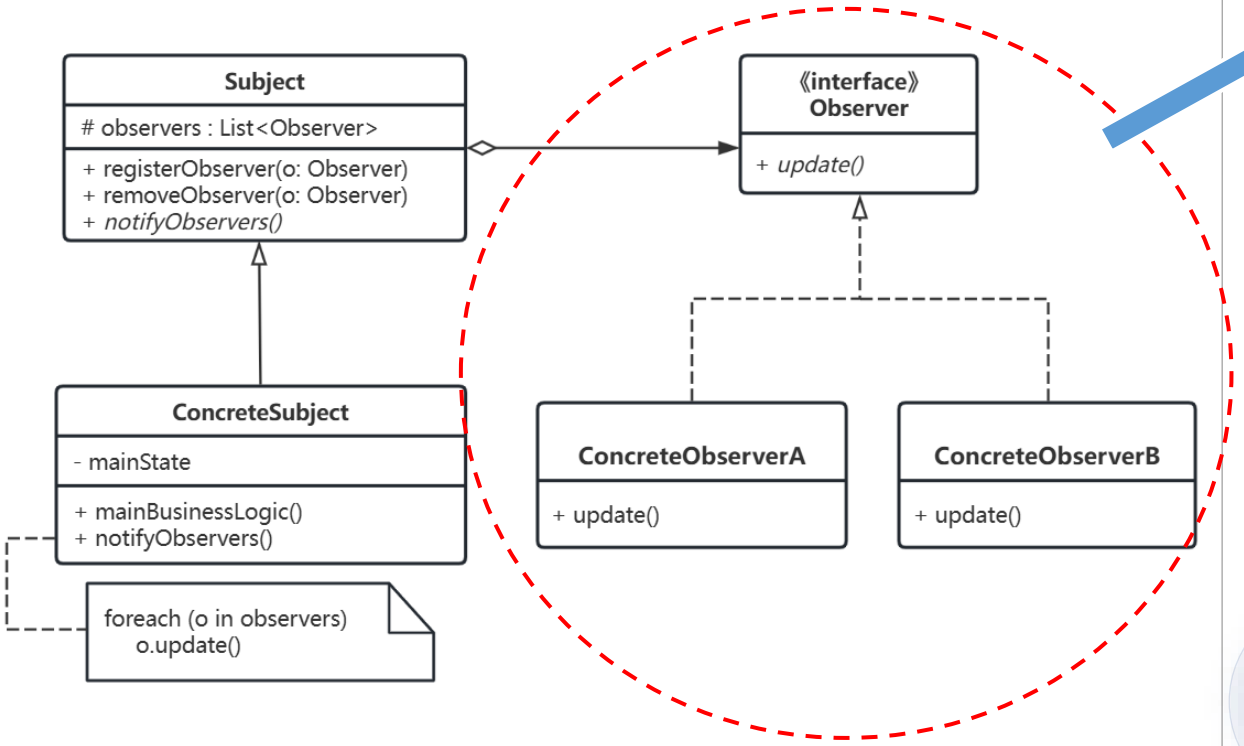


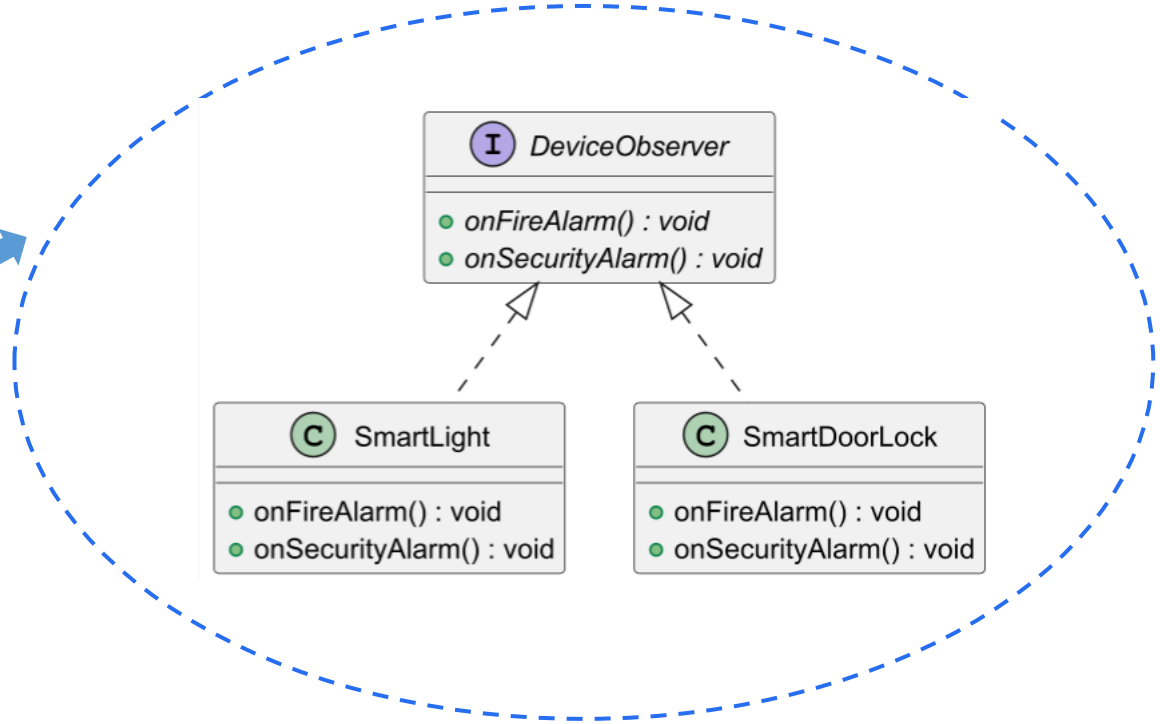
实验步骤：智能家居警告系统举例（观察者模式）

假如我们要实现一个智能家居警告系统：火灾报警时，智能灯开启应急照明，门锁自动解锁以便逃生；安防报警时，智能灯闪烁警示，门锁自动上锁防止入侵。我们该如何用**观察者模式**实现呢？

实验步骤：智能家居警告系统举例（1）

UML 类 图 设 计

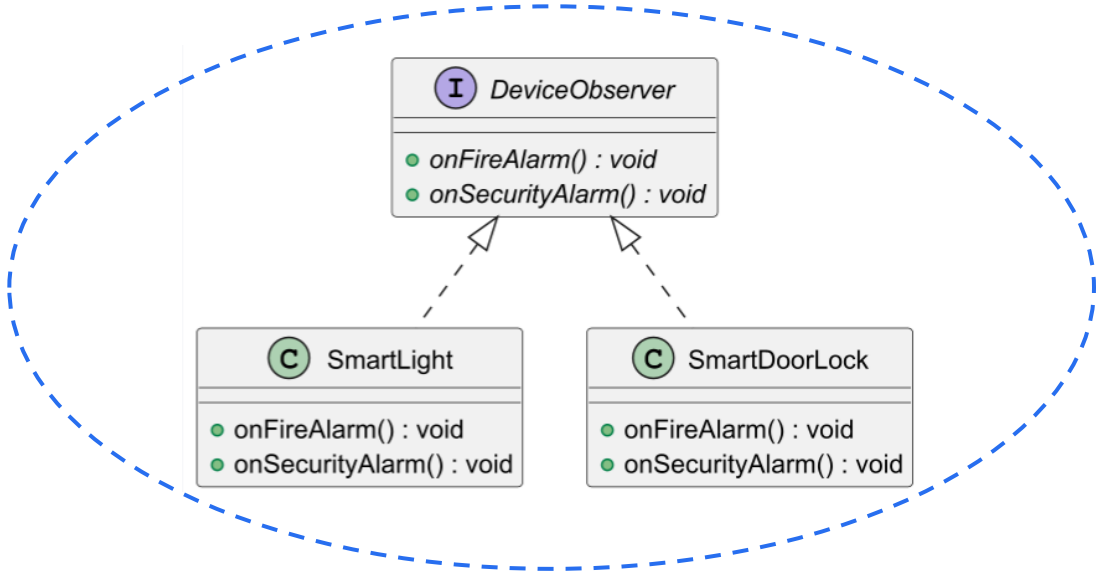




观察者 创建观察者接口和实现该接口的两个具体观察者类

实验步骤：智能家居警告系统举例（2）

代 码 实 现



1 创建观察者接口，充当观察者角色

```java
public interface DeviceObserver {
    // 火灾告警后的反应
    void onFireAlarm();
    // 安防告警后的反应
    void onSecurityAlarm();
}
```

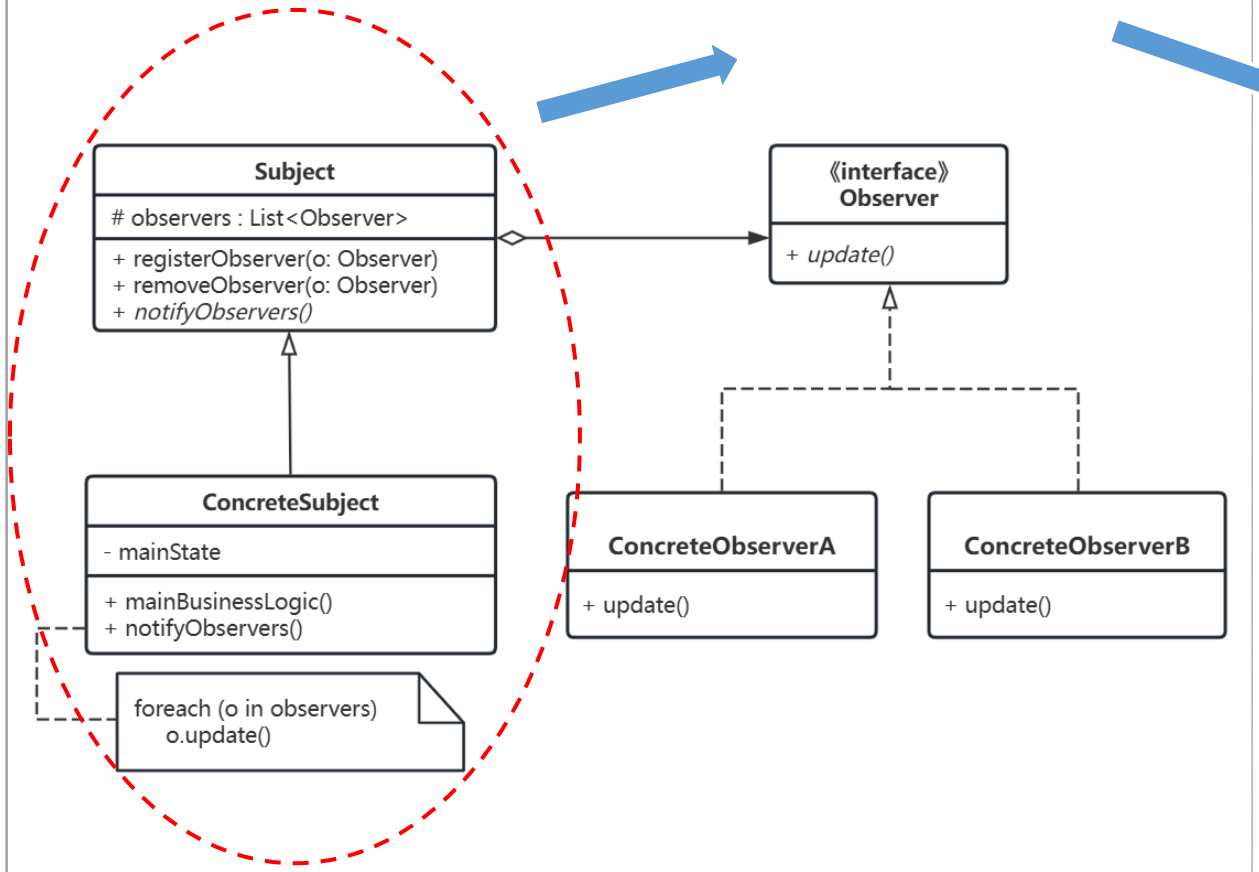
2 创建两个观察者实体类，充当具体观察者角色

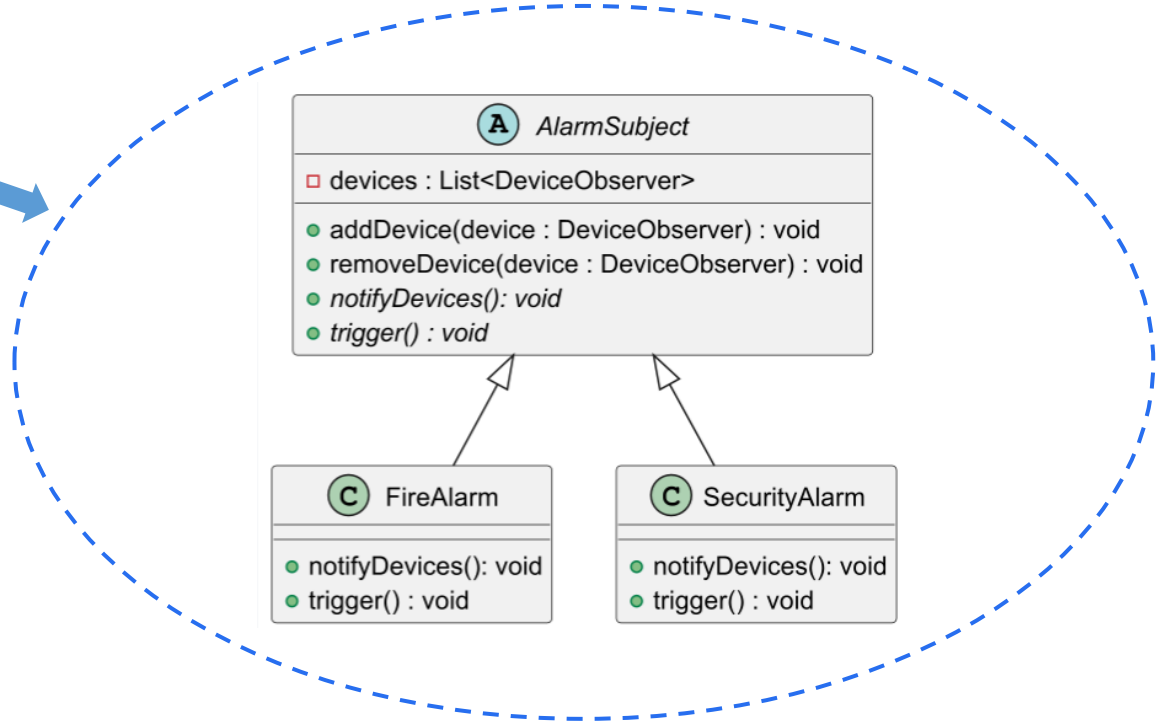
```java
public class SmartLight implements DeviceObserver {
    @Override
    public void onFireAlarm() {
        System.out.println("智能灯：打开应急照明。");
    }
    @Override
    public void onSecurityAlarm() {
        System.out.println("智能灯：开启闪烁警示。");
    }
}
```

```java
public class SmartDoorLock implements DeviceObserver {
    @Override
    public void onFireAlarm() {
        System.out.println("智能门锁：自动解锁，方便人员逃生。");
    }
    @Override
    public void onSecurityAlarm() {
        System.out.println("智能门锁：自动上锁，防止入侵。");
    }
}
```

实验步骤：智能家居警告系统举例（3）

UML 类 图 设 计



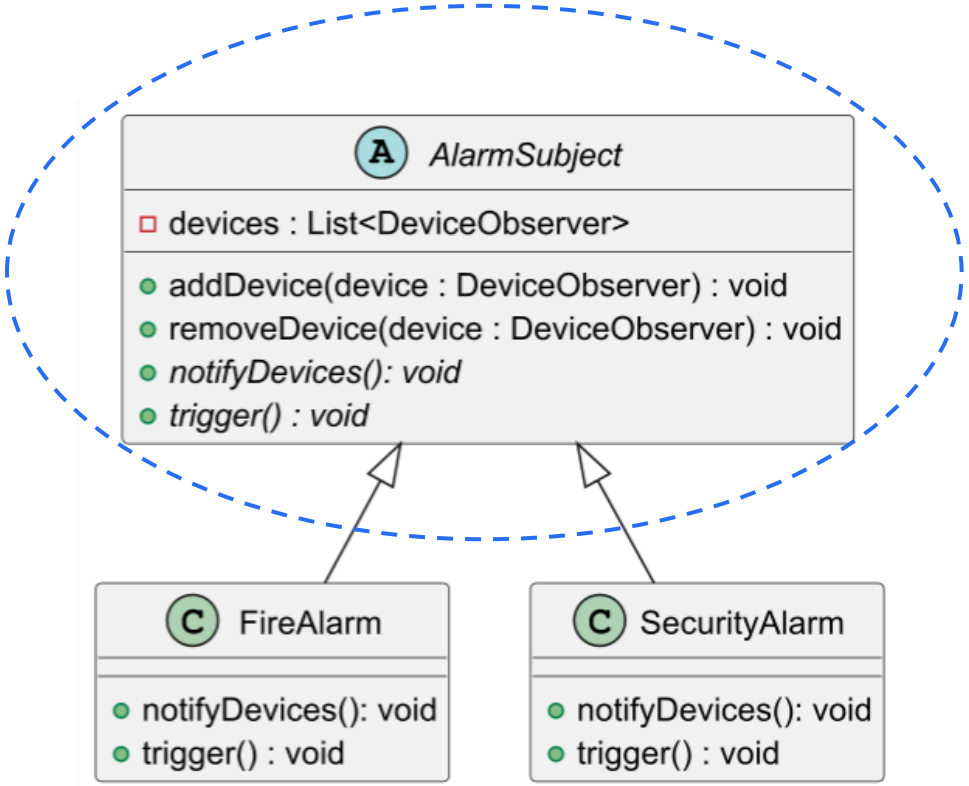


观察目标

创建抽象观察目标类和继承该类的具体观察目标类，当状态改变时，通知所有观察者们

实验步骤：智能家居警告系统举例（4）

代 码 实 现

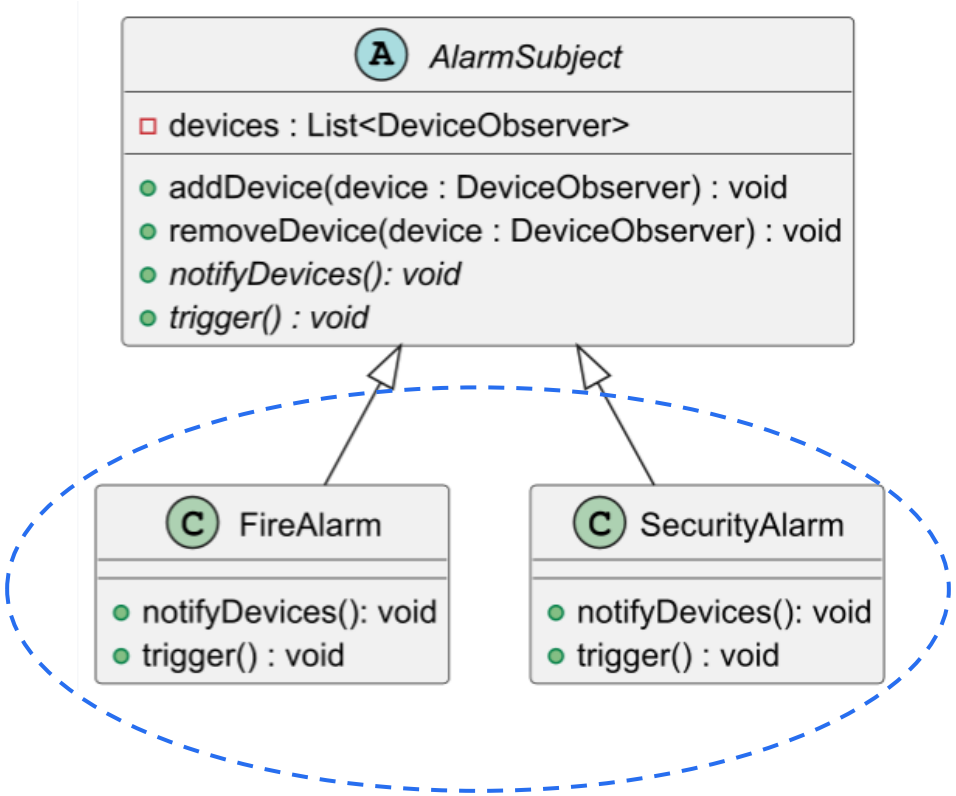


3 创建AlamSubject类，充当抽象观察目标角色

```java
import java.util.ArrayList;
import java.util.List;

public abstract class AlarmSubject {
    protected List<DeviceObserver> devices = new ArrayList<>();

    public void addDevice(DeviceObserver device) {
        devices.add(device);
    }

    public void removeDevice(DeviceObserver device) {
        devices.remove(device);
    }

    Rename usages
    public abstract void notifyDevices();

    public abstract void trigger();
}
```

实验步骤：智能家居警告系统举例（5）

代 码 实 现

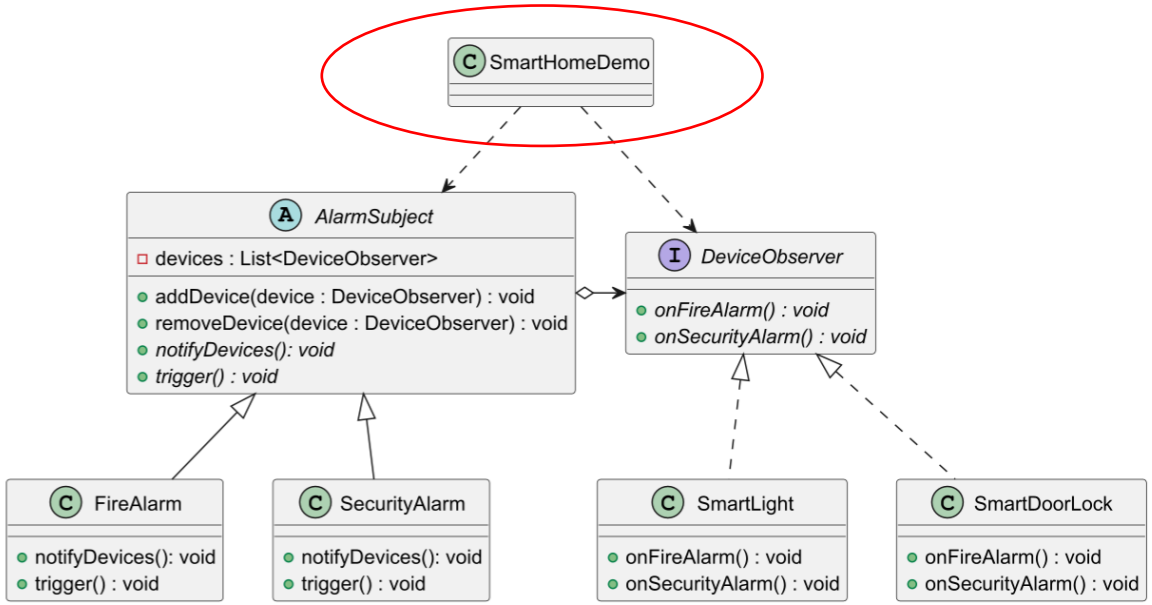


4 创建两个观察目标实体类，充当具体观察目标角色

```java
public class FireAlarm extends AlarmSubject {
    @Override
    public void notifyDevices() {
        for (DeviceObserver device : devices) {
            device.onFireAlarm();
        }
    }
    @Override
    public void trigger() {
        System.out.println("==== 火灾报警器触发! ====");
        notifyDevices();
    }
}
public class SecurityAlarm extends AlarmSubject {
    @Override
    public void notifyDevices() {
        for (DeviceObserver device : devices) {
            device.onSecurityAlarm();
        }
    }
    @Override
    public void trigger() {
        System.out.println("==== 安防报警器触发! ====");
        notifyDevices();
    }
}
```

实验步骤：智能家居警告系统举例（6）

UML 类 图 设 计



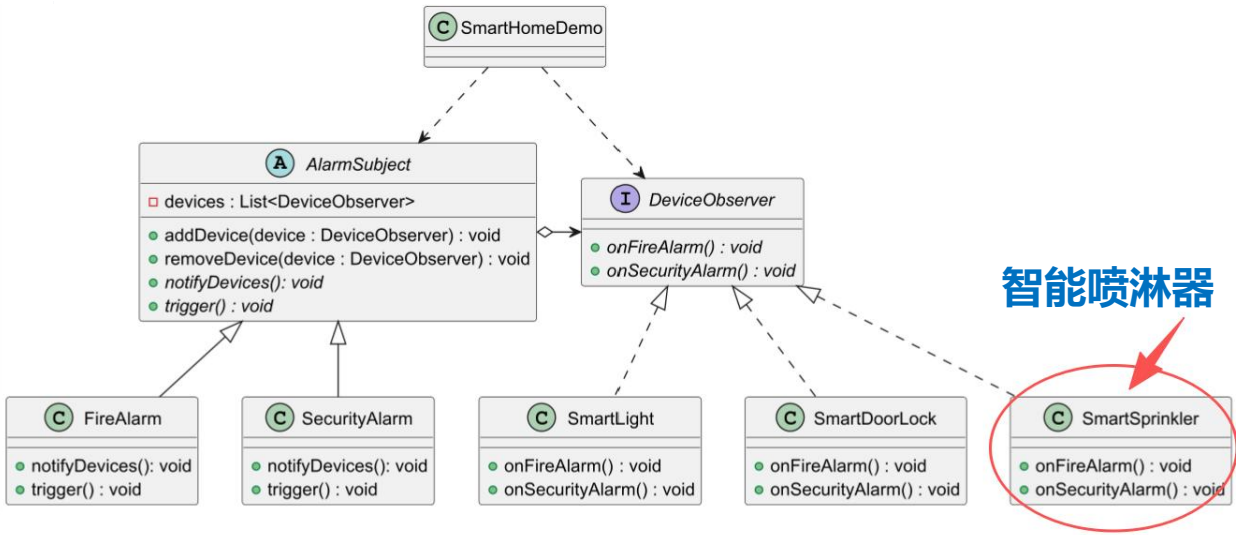
==== 火灾报警器触发！ ====
智能灯：打开应急照明。
智能门锁：自动解锁，方便人员逃生。
==== 安防报警器触发！ ====
智能灯：开启闪烁警示。
智能门锁：自动上锁，防止入侵。

5 客户端注册观察者，并触发观察目标的状态改变

```java
public class SmartHomeDemo {
    public static void main(String[] args) {
        // 1. 初始化两个观察目标
        AlarmSubject fireAlarm = new FireAlarm();
        AlarmSubject securityAlarm = new SecurityAlarm();

        // 2. 初始化两个个观察者
        DeviceObserver light = new SmartLight();
        DeviceObserver doorLock = new SmartDoorLock();

        // 3. 将观察者注册到观察目标中
        fireAlarm.addDevice(light);
        fireAlarm.addDevice(doorLock);

        securityAlarm.addDevice(light);
        securityAlarm.addDevice(doorLock);

        // 4. 触发告警
        fireAlarm.trigger();
        securityAlarm.trigger();
    }
}
```

请思考：如何添加一个智能喷淋器?

UML 类 图 设 计



==== 火灾报警器触发！ ====
智能灯：打开应急照明。
智能门锁：自动解锁，方便人员逃生。
智能喷淋器：启动自动喷水灭火。
==== 安防报警器触发！ ====
智能灯：开启闪烁警示。
智能门锁：自动上锁，防止入侵。
智能喷淋器：安防报警与灭火无关，保持关闭状态。

```java
public class SmartHomeDemo {
    public static void main(String[] args) {
        // 1. 初始化两个观察目标
        AlarmSubject fireAlarm = new FireAlarm();
        AlarmSubject securityAlarm = new SecurityAlarm();

        // 2. 初始化三个个观察者
        DeviceObserver light = new SmartLight();
        DeviceObserver doorLock = new SmartDoorLock();
        DeviceObserver sprinkler = new SmartSprinkler();
        // 3. 将观察者注册到观察目标中
        fireAlarm.addDevice(light);
        fireAlarm.addDevice(doorLock);
        fireAlarm.addDevice(sprinkler);

        securityAlarm.addDevice(light);
        securityAlarm.addDevice(doorLock);
        securityAlarm.addDevice(sprinkler);
        // 4. 触发告警
        fireAlarm.trigger();
        securityAlarm.trigger();

    }
}
```

开闭原则

实验步骤：难度选择场景分析（1）

难度选择场景分析

用户进入游戏界面后，可选择简单、普通和困难三种游戏难度。选择完成后，系统将加载该难度对应的专属地图，并进行相应的初始参数设定。

实验步骤：难度选择场景分析（2）

➢ 三种难度的游戏行为存在**明显差异**：

	初始设定影响因素项
1	屏幕中出现的敌机最大数量
2	不同类型敌机的出现概率
3	英雄机的射击周期
4	敌机的射击周期
5	敌机产生周期
6	Boss 敌机产生阈值

游戏难度	随游戏时间逐步提升的影响因素
简单难度	难度不随游戏时间变化，全程保持初始设定
普通难度	敌机产生周期、敌机速度、敌机血量
困难难度	敌机产生周期、敌机速度、敌机血量、英雄机射击周期、敌机射击周期

游戏难度	Boss 机相关设定
简单难度	无法生成 Boss 敌机
普通难度	每次召唤 Boss 机时，不改变 Boss 机血量
困难难度	每次召唤 Boss 机时，提升 Boss 机血量

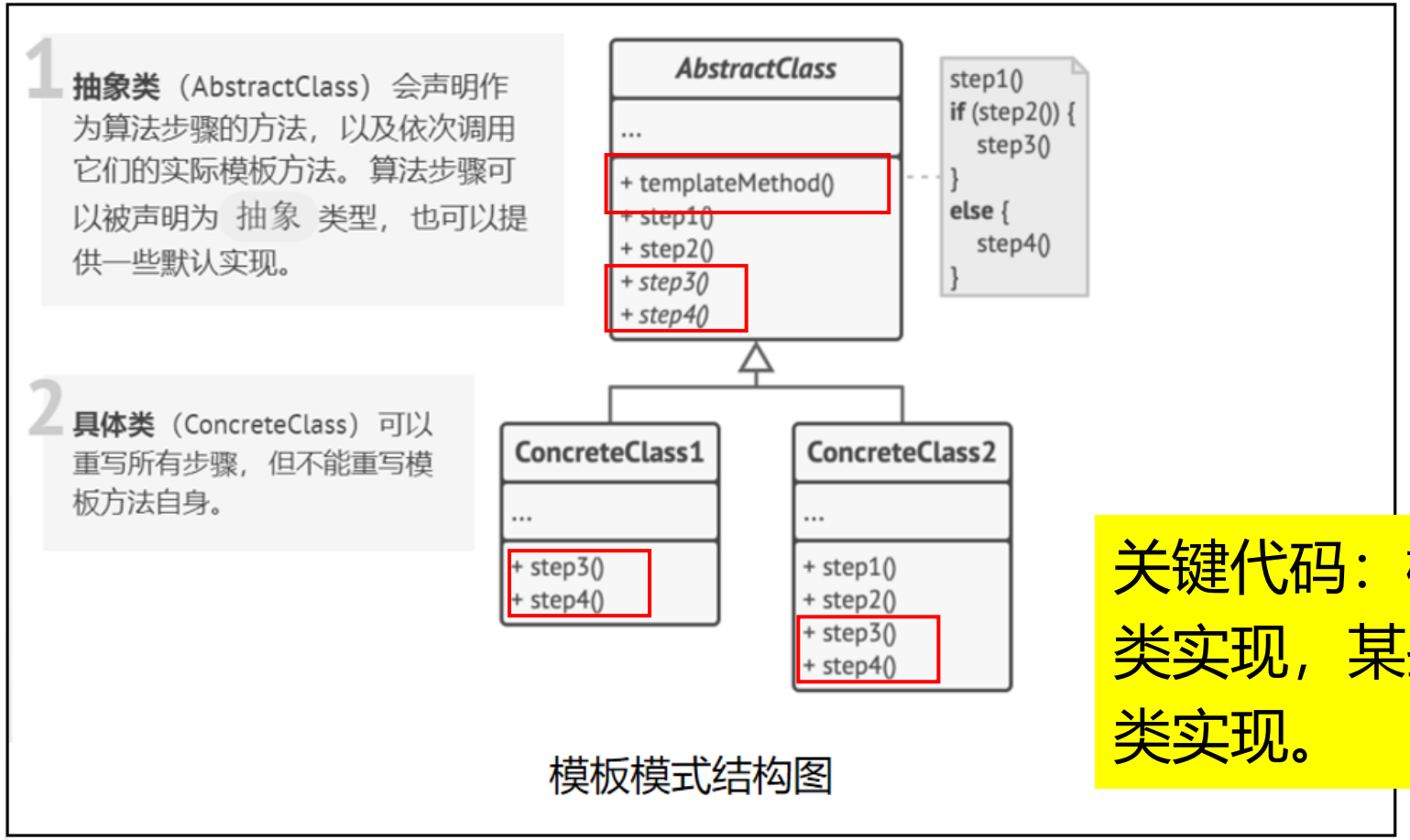
实验步骤：难度选择场景分析（3）

请思考：

1. 三种游戏难度有哪些共性的地方？哪些不同的地方？
2. 若要增加一种新的游戏难度，需要改动哪些地方？
3. 如何实现代码复用？

实验步骤：模板模式结构图

模板模式（Template Pattern）是一种行为型设计模式， 它在抽象类中定义了一个算法的框架， 允许子类在不修改结构的情况下重写算法的特定步骤。



模板模式结构图

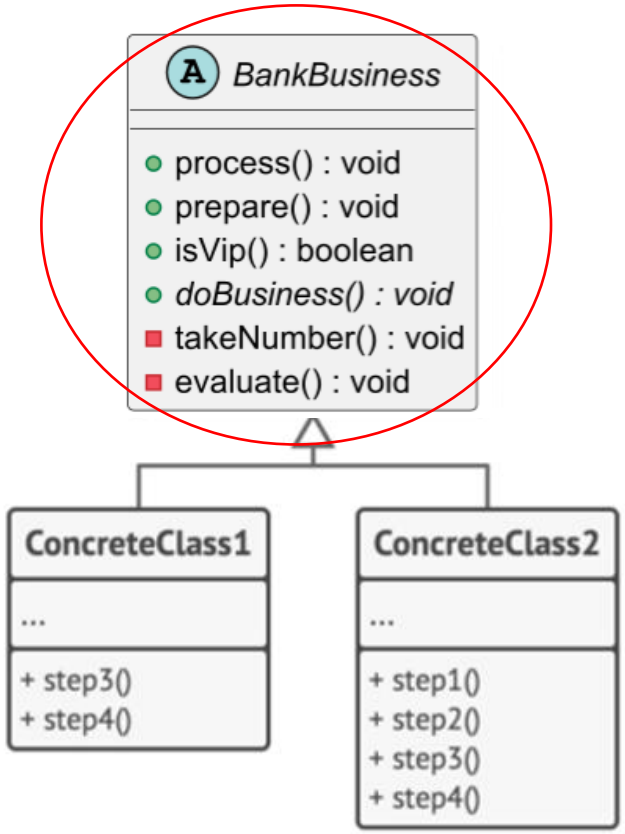
关键代码：模板方法在抽象类实现，某些特定步骤在子类实现。

实验步骤：银行业务办理举例（模板模式）

在银行办理业务时，有**一套标准的流程**：取号、排队、窗口办理、服务评价。其中“取号”和“服务评价”对所有客户都是固定的。但“排队”和“窗口办理”的内容取决于具体的业务类型（如存款或理财）。我们该如何用**模板模式**实现呢？

实验步骤：银行业务办理举例（1）

UML 类 图 设 计



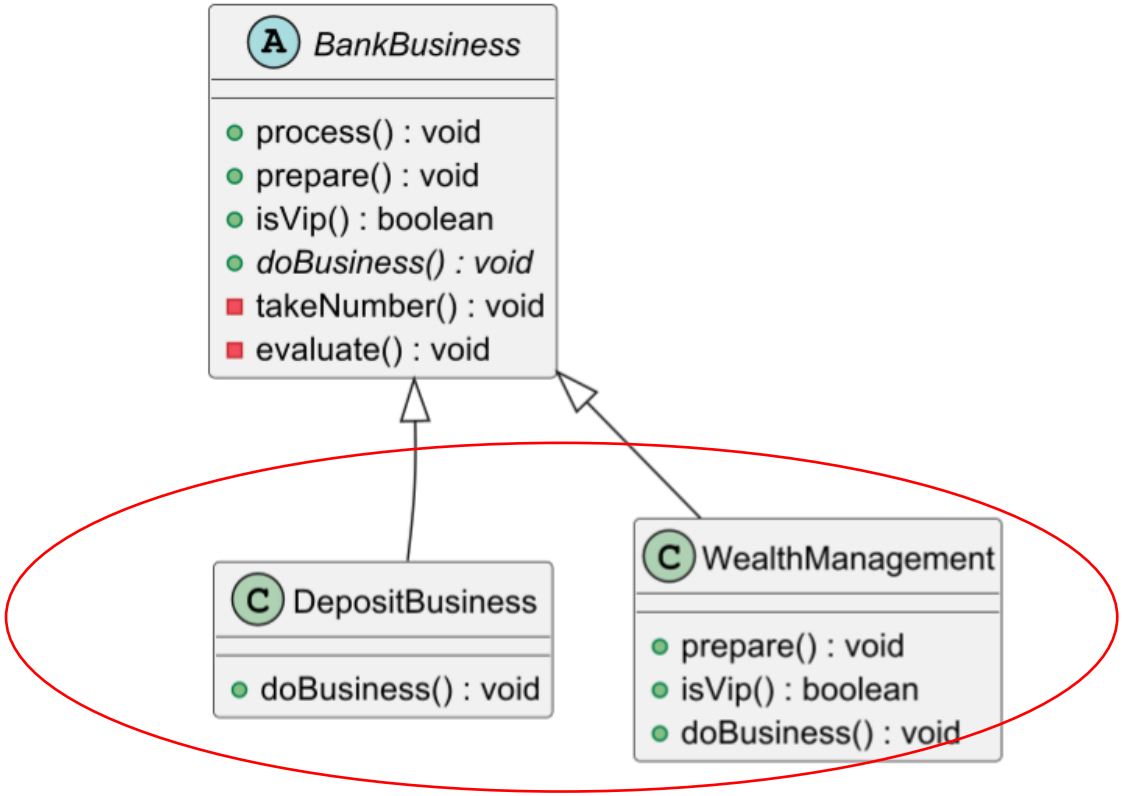
1 创建BankBusiness 抽象类，负责定义业务办理的模板方法，该方法被设置为 final

```java
public abstract class BankBusiness {
    // 模板方法：定义了办理业务的固定骨架
    public final void process() {
        prepare(); // 钩子方法1
        takeNumber(); // 固定步骤
        // 逻辑钩子判断
        if (isVip()) {
            System.out.println("VIP通道：无需等待，直接前往窗口。");
        } else {
            System.out.println("普通通道：请在休息区等候叫号...");
        }
        doBusiness(); // 抽象方法：具体业务内容
        evaluate(); // 固定步骤
    }
    // 钩子方法1：默认不需要准备材料，子类可挂载
    public void prepare() {}
    // 钩子方法2：默认不是VIP，子类可重写返回逻辑
    public boolean isVip() {return false; }
    // 固定步骤：取号
    private void takeNumber() {System.out.println("第1步：取号排队。");}
    // 抽象方法：子类必须实现具体的业务逻辑
    public abstract void doBusiness();
    // 固定步骤：评价
    private void evaluate() {System.out.println("最后一步：反馈评价。");}
}
```

模板方法

实验步骤：银行业务办理举例（2）

UML 类 图 设 计

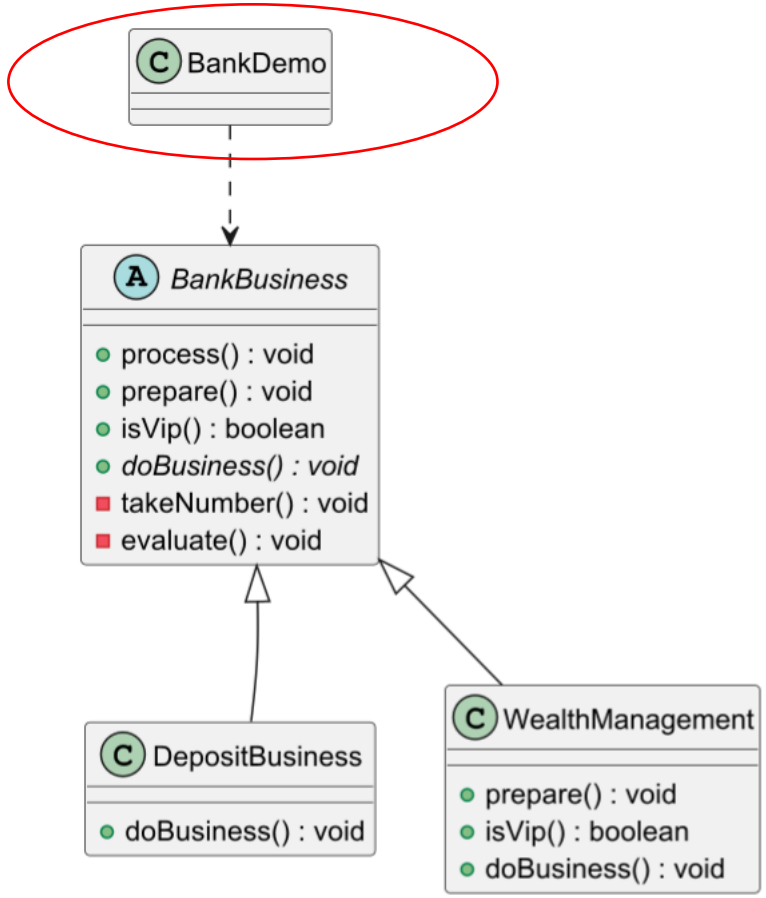


2 扩展该抽象类的两个实体类，它们重写了抽象类的某些方法

```java
public class DepositBusiness extends BankBusiness {
    @Override
    public void doBusiness() {
        System.out.println("核心业务：办理现金存款 5000 元。");
    }
    // 采用父类默认的钩子：不准备材料，非VIP
}

public class WealthManagement extends BankBusiness {
    @Override
    public void prepare() {
        System.out.println("前置准备：填写《风险测评问卷》和《大额交易申报单》。");
    }
    @Override
    public boolean isVip() {
        // 重写钩子，使其走 VIP 逻辑分支
        return true;
    }
    @Override
    public void doBusiness() {
        System.out.println("核心业务：购买 100 万元大额理财产品。");
    }
}
```

实验步骤：银行业务办理举例（3）

UML 类 图 设 计



3 使用BankDemo 的模板方法 process() 来演示模板模式的用法

```
public class BankDemo {
    public static void main(String[] args) {

        System.out.println("=== 场景一：用户1办存款 ===");
        BankBusiness user1 = new DepositBusiness();
        user1.process();

        System.out.println("\n=== 场景二：用户2办理财 ===");
        BankBusiness user2 = new WealthManagement();
        user2.process();
    }
}
```

=== 场景一：用户1办存款 ===
第1步：取号排队。
普通通道：请在休息区等候叫号...
核心业务：办理现金存款 5000 元。
最后一步：反馈评价。
=== 场景二：用户2办理财 ===
前置准备：填写《风险测评问卷》和《大额交易申报单》。
第1步：取号排队。
VIP通道：无需等待，直接前往窗口。
核心业务：购买 100 万元大额理财产品。
最后一步：反馈评价。

本次迭代开发目标（1）

UML类图设计

① 结合实例，按照规范绘制观察者模式与模板模式的结构类图

设计模式应用

② 完成代码重构，使用观察者模式实现炸弹、冰冻道具，使用模板模式实现三种游戏难度

道具效果

③ 确保英雄机碰撞炸弹、冰冻道具后，可自动触发对应生效效果；不同道具对不同类型敌机的作用效果存在显著差异，具体效果详见炸弹、冰冻道具场景分析

注：实现模板模式时，要求将 Game 类的 action 方法重构为模板方法

本次迭代开发目标（2）

难度选择功能

④ 设计并实现“简单”、“普通”和“困难”三种游戏难度的选择功能，三种难度的游戏需符合场景设定，存在明显差异，具体要求详见难度选择场景分析

Boss敌机控制

⑤ 简单难度无法生成Boss敌机；普通难度每次召唤Boss机不改变其血量；困难难度每次召唤Boss机提升其血量

难度递进功能

⑥ 实现普通、困难两种模式的难度递进逻辑，确保游戏难度随游戏时间逐步递增，具体递增规则详见难度选择场景分析，需在控制台输出相应信息。

期末提交要求

➢ 提交内容（截止日期参考平台发布）

① **整个项目压缩包：**将整个项目压缩为zip格式提交，需包含全部代码、UML类图等相关文件

② **实验报告：**严格按照给定的实验报告模板完成撰写

③ **选做内容：**录制一段系统演示视频（时长需小于2分钟），视频中需清晰展示所有功能点及设计亮点。

注：创新功能有加分，请在实验报告和视频中展示，总分不超过100分。

同学们，请开始实验吧！

THANK YOU